

|                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>lru-cache</div> <div> <p>A cache object that deletes the least-recently-used items.</p> <p>Specify a max number of the most recently used items that you want to keep, and this cache will keep that many of the most recently accessed items.</p> <p>This is not primarily a TTL cache, and does not make strong TTL guarantees. There is no preemptive pruning of expired items by</p> </div> <div>1</div> | <div> <p>default, but you <i>may</i> set a TTL on the cache or on a single set. If you do so, it will treat expired items as missing, and delete them when fetched. If you are more interested in TTL caching than LRU caching, check out <a href="#">@isaacs/ttlcache</a>.</p> <p>As of version 7, this is one of the most performant LRU implementations available in JavaScript, and supports a wide diversity of use cases. However, note that using some of the features will necessarily impact performance, by causing the cache to have to</p> </div> <div>2</div> | <div> <p>do more work. See the “Performance” section below.</p> <div>Installation</div> <pre>npm install lru-cache --save</pre> <div>Usage</div> <pre>// hybrid module, either works import { LRUCache } from 'lru-cache' // or:</pre> </div> <div>3</div>                                                                                                                                                                                                                                  | <div> <pre>const { LRUCache } = require('lru-cache') // or in minified form for web browsers: import { LRUCache } from 'http://unpkg.com/lru-cache@9/dist/mjs/index.min.mjs'</pre> <pre>// At least one of 'max', 'ttl', or 'maxSize' is required, to prevent // unsafe unbounded storage.</pre> </div> <div>4</div>                                                                                                                                                                                                         |
| <div>8</div> <pre>updateAgeOnGet: false, updateAgeOnHas: false, allowStale: false, // return stale items before // removing from cache? ttl: 1000 * 60 * 5, // how long to live in ms }, logInsertionOrWhatever(key, value)</pre>                                                                                                                                                                                 | <div>7</div> <pre>// are evicted from the cache dispose: (value, key, reason) =&gt; {   freeFromMemoryOrWhatever(value) }, // for use when you need to know // that an item is being inserted // note that this does NOT allow you // to prevent the insertion, // it just allows you to know about it. onInsert: (value, key, reason) =&gt; {</pre>                                                                                                                                                                                                                       | <div>9</div> <pre>const options = {   max: 500,   // for use with tracking overall   // storage size   maxSize: 5000,   sizeCalculation: (value, key) =&gt; {     return 1   },   // for use when you need to clean up   // something when objects</pre>                                                                                                                                                                                                                                    | <div>5</div> <pre>// // In most cases, it's best to specify // a max for performance, so all // the required memory allocation is // done up-front. // // All the other options are optional, // see the sections below for // documentation on what each one // does. Most of them can be // overridden for specific items in   get()/set()</pre>                                                                                                                                                                           |
| <pre>// async method to use for // cache.fetch(), for // stale-while-revalidate type of // behavior fetchMethod: async (key,   staleValue, { options,   signal, context }) =&gt; {}, }  const cache = new LRUCache(options)</pre> <div>9</div>                                                                                                                                                                    | <pre>cache.set('key', 'value') cache.get('key') // "value"  // non-string keys ARE fully supported // but note that it must be THE SAME // object, not // just a JSON-equivalent object. var someObject = { a: 1 } cache.set(someObject, 'a value') // Object keys are not toString()-ed cache.set('[object Object]', 'a   different value')</pre> <div>10</div>                                                                                                                                                                                                           | <pre>assert.equal(cache.get(someObject),   'a value') // A similar object with same keys/ // values won't work, // because it's a different object // identity assert.equal(cache.get({ a: 1 }),   undefined)  cache.clear() // empty the cache</pre> <div>11</div>                                                                                                                                                                                                                         | <div> <p>If you put more stuff in the cache, then less recently used items will fall out. That’s what an LRU cache is.</p> <p>For full description of the API and all options, please see <a href="#">the LRUCache typedocs</a></p> <div>Storage Bounds Safety</div> <p>This implementation aims to be as flexible as possible, within the limits of safe memory consumption and optimal performance.</p> <p>At initial object creation, storage is allocated for max items. If max is set to zero,</p> </div> <div>12</div> |
| <div>16</div> <pre>// a storage-unbounded ttl cache that // is not an lru-cache const cache = {   data: new Map(),   timers: new Map(),   set: (k, v, ttl) =&gt; {     if (cache.timers.has(k)) {       clearTimeout(cache.timers.get(k))     }     cache.timers.set(       k, </pre>                                                                                                                             | <div>15</div> <pre>In this case, a warning is printed to standard error. Future versions may require the use of ttlAutopurge if max and maxSize are not specified. If you truly wish to use a cache that is bound <i>only</i> by TTL expiration, consider using a Map object, and calling setTimeout to delete entries when they expire. It will perform much better than an LRU cache. Here is an implementation you may use, under the same <a href="#">license</a> as this package:</pre>                                                                               | <div>14</div> <pre>cache.set(). The size of every item <i>must</i> be a positive integer. If neither max nor maxSize are set, then ttl tracking must be enabled. Note that, even when tracking item ttl, items are <i>not</i> preemptively deleted when they become stale, unless ttlAutopurge is enabled. Instead, they are only purged the next time the key is requested. Thus, if ttlAutopurge, max, and maxSize are all not set, then the cache will potentially grow unbounded.</pre> | <div>13</div> <pre>then some performance is lost, and item count is unbounded. Either maxSize or ttl <i>must</i> be set if max is not specified. If maxSize is set, then this creates a safe limit on the maximum storage consumed, but without the performance benefits of pre- allocation. When maxSize is set, every item <i>must</i> provide a size, either via the sizeCalculation method provided to the constructor, or via a size or sizeCalculation option provided to</pre>                                        |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>      setTimeout(() =&gt;         cache.delete(k), ttl),     )     cache.data.set(k, v)   },   get: k =&gt; cache.data.get(k),   has: k =&gt; cache.data.has(k),   delete: k =&gt; {     if (cache.timers.has(k)) {       clearTimeout(cache.timers.get(k))     }     cache.timers.delete(k)   } }</pre>                                                                                                                                                                                                                | <pre>      return cache.data.delete(k)     },     clear: () =&gt; {       cache.data.clear()       for (const v of         cache.timers.values()) {         clearTimeout(v)       }       cache.timers.clear()     },   }, }</pre>                                                                                                                                                                                                                                                                                                                                                                                    | <p>If that isn't to your liking, check out <a href="#">@isaacs/ttlcache</a>.</p> <h3>Storing Undefined Values</h3> <p>This cache never stores undefined values, as undefined is used internally in a few places to indicate that a key is not in the cache.</p> <p>You may call <code>cache.set(key, undefined)</code>, but this is just an alias for <code>cache.delete(key)</code>. Note that this has the</p>                                                                                                                                                           | <p>effect that <code>cache.has(key)</code> will return <i>false</i> after setting it to undefined.</p> <p><code>cache.set(myKey, undefined)</code><br/><code>cache.has(myKey) // false!</code></p> <p>If you need to track undefined values, and still note that the key is in the cache, an easy workaround is to use a sigil object of your own.</p>                                                                                                               |
| 17                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 18                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 19                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 20                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <p>Also beware of "Just So" stores regarding performance. Garbage collection of large (especially: deep) object graphs can be incredibly costly, with several "tipping points" where it increases exponentially. As a result, putting that off until later can make it much worse, and less predictable. If a library</p> <pre>const get = k =&gt; myCache[k]  set(({, 'please hang onto this for me' )   set('object object', 'oopsie')</pre>                                                                               | <p>is highly optimized for objects with keys that are short strings, especially integer numeric strings. Thus any benchmark which tests <i>solely</i> using numbers as keys will tend to find that an object-based approach performs the best.</p> <p>Note that coercing <i>anything</i> to strings to use as object keys is unsafe, unless you can be 100% certain that no other type of value will be used. For example:</p> <pre>const myCache = {} const set = (k, v) =&gt; { myCache[k] = v }</pre>                                                                                                              | <p>As of January 2022, version 7 of this library is one of the most performant LRU cache implementations in JavaScript. Benchmarks can be extremely difficult to get right. In particular, the performance of set/get/delete operations on objects will vary <i>wildly</i> depending on the type of key used. V8</p> <h3>Performance</h3> <pre>return v === undefinedValue ?   undefined : v }</pre>                                                                                                                                                                       | <pre>import { LRUCache } from 'lru-cache' const undefinedValue =   Symbol('undefined') const cache = new LRUCache(...) const mySet = (key, value) =&gt;   cache.set(key, value ===     undefined ? undefinedValue :     value) const myGet = (key, value) =&gt; {   const v = cache.get(key)</pre>                                                                                                                                                                   |
| 24                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 23                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 22                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 21                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <p>performs well, but only in a scenario where the object graph is kept shallow, then that won't help you if you are using large objects as keys.</p> <p>In general, when attempting to use a library to improve performance (such as a cache like this one), it's best to choose an option that will perform well in the sorts of scenarios where you'll actually use it.</p> <p>This library is optimized for repeated gets and minimizing eviction time, since that is the expected need of a LRU. Set operations are</p> | <p>somewhat slower on average than a few other options, in part because of that optimization. It is assumed that you'll be caching some costly operation, ideally as rarely as possible, so optimizing set over get would be unwise.</p> <p>If performance matters to you:</p> <ol style="list-style-type: none"><li>If it's at all possible to use small integer values as keys, and you can guarantee that no other types of values will be used as keys, then do that, and use a cache such as <a href="#">lru-fast</a>, or <a href="#">mnemonist's LRUCache</a> which uses an Object as its data store.</li></ol> | <ol style="list-style-type: none"><li>Failing that, if at all possible, use short non-numeric strings (ie, less than 256 characters) as your keys, and use <a href="#">mnemonist's LRUCache</a>.</li><li>If the types of your keys will be anything else, especially long strings, strings that look like floats, objects, or some mix of types, or if you aren't sure, then this library will work well for you.</li></ol> <p>If you do not need the features that this library provides (like asynchronous fetching, a variety of TTL staleness options, and so on),</p> | <p>then <a href="#">mnemonist's LRUMap</a> is a very good option, and just slightly faster than this module (since it does considerably less).</p> <ol style="list-style-type: none"><li>Do not use a dispose function, size tracking, or especially ttl behavior, unless absolutely needed. These features are convenient, and necessary in some use cases, and every attempt has been made to make the performance impact minimal, but it isn't nothing.</li></ol> |
| 25                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 26                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 27                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 28                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <p>cache.fetch() return type is now Promise&lt;V   undefined&gt; instead of void&lt;. This is an irrelevant change practically speaking, but can require changes for TypeScript users. For more info, see the <a href="#">change log</a>.</p> <h3>Breaking Changes in Version 10</h3>                                                                                                                                                                                                                                        | <p>Keys and values must not be null or undefined.</p> <ul style="list-style-type: none"><li>Minified export available at 'lru-cache/context', for both CJS and MJS builds.</li></ul> <h3>Breaking Changes in Version 9</h3> <ul style="list-style-type: none"><li>Named export only, no default export. AbortController polyfill returned, albeit with a warning when used.</li></ul>                                                                                                                                                                                                                                 | <p>The fetchContext option was renamed to context, and may no longer be set on the cache instance itself.</p> <ul style="list-style-type: none"><li>Rewritten in TypeScript, so pretty much all the types moved around a lot.</li><li>The AbortController/AbortSignal polyfill was removed. For this reason, <b>Node version 16.14.0 or higher is now required</b>.</li><li>Internal properties were moved to actual private class properties.</li></ul>                                                                                                                   | <p>This library changed to a different algorithm and internal data structure in version 7, yielding significantly better performance, albeit with some subtle changes as a result. If you were relying on the internals of LRUCache in version 6 or before, it probably will not work in version 7 and above.</p> <h3>Breaking Changes in Version 7</h3>                                                                                                             |
| 32                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 31                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 30                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 29                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |